

附录：课程论文清单

概述

本清单收录了与 AI 编程代理 (AI Programming Agents) 相关的核心论文，按技术层次从底层到上层组织为六个类别：**基石层、反思层、编程代理层、搜索策略层、多代理协作层、模型底座层**。每一层的论文都为上层能力提供了理论或技术基础。

本清单共包含 **18 篇核心论文**（分布在 6 个层次中）和 **15+ 篇扩展阅读**，覆盖了从 2021 年至 2024 年间该领域最重要的研究进展。

使用建议： - 每篇论文标注了与课程模块的对应关系，方便按课程进度同步阅读 - 核心贡献部分帮助你快速判断是否需要精读全文 - “推荐阅读章节”指出了每篇论文中最值得深入研读的部分 - 带 □ 标记的论文为必读论文（共 9 篇），其余为推荐阅读 - 末尾附有术语对照表和阅读笔记模板，辅助高效阅读

层次关系概览：

多代理协作层 (Multi-Agent)	
搜索策略层 (Search Strategy)	
编程代理层 (Programming Agent)	
反思层 (Reflection)	
基石层 (Foundation)	
模型底座层 (Model Foundation)	

每一层都建立在下层的能力之上：模型底座提供基础的代码理解和指令跟随能力；基石层定义推理、行动和工具使用的核心范式；反思层赋予自我改进能力；编程代理层将这些能力应用于软件工程；搜索策略层实现更系统的解空间探索；多代理协作层则处理大规模任务的分工与协调。

基石层（Foundation Layer）

基石层论文奠定了 LLM 代理的三大核心能力：**推理（Reasoning）**、**行动（Acting）**、**工具使用（Tool Use）**。这些工作定义了当前代理系统的基本范式。

论文	作者	年份	发表 Venue	核心贡献	推荐阅读章节
□ Chain-of-Thought Prompting Elicits Reasoning in Large Language Models	Wei et al.	2022	NeurIPS 2022	提出思维链提示方法，使 LLM 具备逐步推理能力	§ 2, § 3, Figure 1
□ ReAct: Synergizing Reasoning and Acting in Language Models	Yao et al.	2022	ICLR 2023	将推理与行动交织进行，奠定代理循环范式	§ 3, § 4, Figure 1-2
□ Toolformer: Language Models Can Teach Themselves to Use Tools	Schick et al.	2023	NeurIPS 2023	让模型自主学习调用外部工具	§ 2, § 3, Table 1-2

论文摘要

1. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models

- **完整引用：** Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., & Zhou, D. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. NeurIPS 2022.
- **链接：** <https://arxiv.org/abs/2201.11903>

- **核心贡献：** 本文提出了思维链（Chain-of-Thought, CoT）提示方法，通过在少样本提示中加入中间推理步骤，显著提升了大语言模型在算术推理、常识推理和符号推理任务上的表现。这一发现揭示了模型规模与涌现推理能力之间的关系——CoT 效果在模型参数达到约 100B 时显著涌现。该方法简单但影响深远，为后续所有基于 LLM 的代理系统奠定了“让模型思考”的核心范式。
- **关键图表：** Figure 1（CoT 示例对比）、Table 1（不同规模模型的性能对比）
- **课程关联：** 模块 1（代理基础）——理解代理为何需要逐步推理，以及如何通过提示工程激发模型能力。

2. ReAct: Synergizing Reasoning and Acting in Language Models

- **完整引用：** Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2022). ReAct: Synergizing Reasoning and Acting in Language Models. ICLR 2023.
- **链接：** <https://arxiv.org/abs/2210.03629>
- **核心贡献：** ReAct 将推理（Reasoning）和行动（Acting）统一在一个交织框架中：模型在每一步既进行自然语言推理，又执行具体操作（如搜索、查询）。这种“思考—行动—观察”循环模式成为了几乎所有现代 LLM 代理的标准架构。论文证明推理轨迹能帮助模型更好地规划、跟踪和调整行动，而行动结果又能支撑更准确的推理。
- **关键图表：** Figure 1（ReAct 与 CoT、Act-only 的对比）、Figure 2（推理-行动交织示例）
- **课程关联：** 模块 2（代理循环与工具调用）——ReAct 是理解代理循环（Agent Loop）最重要的论文，直接对应代理的核心运行机制。

3. Toolformer: Language Models Can Teach Themselves to Use Tools

- **完整引用：** Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., & Scialom, T. (2023). Toolformer: Language Models Can Teach Themselves to Use Tools. NeurIPS 2023.
 - **链接：** <https://arxiv.org/abs/2302.04761>
 - **核心贡献：** Toolformer 提出了一种自监督方法，让语言模型学会在生成文本的过程中自主决定何时、如何调用外部工具（计算器、搜索引擎、翻译 API 等）。模型通过自我标注训练数据来学习工具调用，无需大量人工标注。这项工作从根本上证明了 LLM 可以成为通用的工具使用者，为后续编程代理的工具集成提供了理论基础。
 - **关键图表：** Figure 1（工具调用注入流程）、Table 1-2（各工具的使用效果）
 - **课程关联：** 模块 2（代理循环与工具调用）——理解代理如何学习使用工具，以及工具调用接口设计的原则。
-

反思层（Reflection Layer）

反思层论文探索了代理的**自我评估与迭代改进能力**。从简单的输出优化到复杂的搜索策略，这些工作让代理具备了“反思—修正—提升”的闭环能力。

论文	作者	年份	发表 Venue	核心贡献	推荐阅读章节
□ Self-Refine: Iterative Refinement with Self-Feedback	Madaan et al.	2023	NeurIPS 2023	提出单模型自我反馈迭代优化框架	§ 2, § 3, Figure 1-2
□ Reflexion: Language Agents with Verbal Reinforcement Learning	Shinn et al.	2023	NeurIPS 2023	将语言反馈作为强化学习信号用于代理改进	§ 3, § 4, Figure 1
Tree of Thoughts: Deliberate Problem Solving with Large Language Models	Yao et al.	2023	NeurIPS 2023	将推理扩展为树状搜索结构	§ 2, § 3, Figure 1-2

论文摘要

1. Self-Refine: Iterative Refinement with Self-Feedback

- **完整引用：** Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegreffe, S., Alon, U., Dziri, N., Prabhumoye, S., Yang, Y., Gupta, S., Majumder, B.P., Hermann, K.M., Welleck, S., Yazdanbakhsh, A., & Clark, P. (2023). Self-Refine: Iterative Refinement with Self-Feedback. NeurIPS 2023.
- **链接：** <https://arxiv.org/abs/2303.17651>
- **核心贡献：** Self-Refine 提出了一种简洁而强大的迭代优化框架：同一个 LLM 交替扮演“生成者”和“评审者”两个角色，先生成初始输出，再对其进行自我反馈，最后根据反馈优化输出，如此循环直至满意。该方法无需额外训练或外部标注，在

代码优化、数学推理、对话生成等多个任务上均表现出显著提升。这一工作为编程代理的自我调试和代码优化提供了核心方法论。

- **关键图表：** Figure 1（迭代优化流程）、Figure 2（各任务的多轮改进曲线）
- **课程关联：** 模块 3（反思与迭代改进）——Self-Refine 是理解代理如何通过自我评估不断提升输出质量的起点。

2. Reflexion: Language Agents with Verbal Reinforcement Learning

- **完整引用：** Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., & Yao, S. (2023). Reflexion: Language Agents with Verbal Reinforcement Learning. NeurIPS 2023.
- **链接：** <https://arxiv.org/abs/2303.11366>
- **核心贡献：** Reflexion 将传统强化学习中的标量奖励信号替换为自然语言形式的“语言反馈”，代理在每次任务尝试失败后生成文字形式的反思总结，存入长期记忆，在下一次尝试中参考这些经验教训。这种方法在编程任务（HumanEval）上实现了从 baseline 到 91% 的巨大提升。Reflexion 证明了代理可以通过积累语言形式的经验来持续学习，无需更新模型参数。
- **关键图表：** Figure 1（Reflexion 整体架构）、Table 1（编程任务上的结果对比）
- **课程关联：** 模块 3（反思与迭代改进）——理解代理如何从失败中学习，以及经验记忆在代理系统中的作用。

3. Tree of Thoughts: Deliberate Problem Solving with Large Language Models

- **完整引用：** Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T.L., Cao, Y., & Narasimhan, K. (2023). Tree of Thoughts: Deliberate Problem Solving with Large Language Models. NeurIPS 2023.
 - **链接：** <https://arxiv.org/abs/2305.10601>
 - **核心贡献：** Tree of Thoughts (ToT) 将 LLM 的推理过程从线性链式结构扩展为树状搜索结构。在每个推理步骤，模型可以生成多个候选“思维”，通过自我评估对各分支进行打分，然后使用 BFS 或 DFS 策略探索最有前景的路径。这种方法在需要前瞻规划和回溯的复杂问题（如 24 点游戏、创意写作）上大幅优于线性 CoT。ToT 开创了“推理时计算扩展”（test-time compute scaling）的研究方向。
 - **关键图表：** Figure 1（线性 vs 树状推理对比）、Figure 2（搜索策略可视化）
 - **课程关联：** 模块 4（搜索与规划策略）——ToT 是连接反思层和搜索策略层的桥梁，理解如何将推理组织为可搜索的结构。
-

编程代理层（Programming Agent Layer）

编程代理层论文聚焦于将 LLM 代理应用于真实软件工程任务——包括 Bug 修复、功能开发、代码生成等。这些工作定义了编程代理的交互接口和评估标准。

论文	作者	年份	发表 Venue	核心贡献	推荐阅读章节
□ SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering	Yang et al.	2024	arXiv	定义了代理-计算机接口（ACI）设计原则	§ 3, § 4, Figure 1-3
□ CodeAct: Executable Code Actions Elicit Better LLM Agents	Wang et al.	2024	arXiv	用可执行代码替代 JSON 作为代理动作空间	§ 2, § 3, Table 1-2
Agentless: Demystifying LLM-based Software Engineering Agents	Xia et al.	2024	arXiv	证明无代理流水线也可取得竞争力	§ 2, § 3, Table 1

论文摘要

1. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering

- **完整引用：** Yang, J., Jimenez, C.E., Wettig, A., Liber, K., Narasimhan, K., & Press, O. (2024). SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. arXiv preprint arXiv:2405.15793.
- **链接：** <https://arxiv.org/abs/2405.15793>

- **核心贡献：** SWE-agent 提出了“代理-计算机接口”（Agent-Computer Interface, ACI）的概念，强调为 LLM 代理设计专用的交互接口与人类使用的 GUI 或原始终端同等重要。通过精心设计的文件浏览、搜索、编辑等命令接口，SWE-agent 在 SWE-bench 上取得了领先性能。该论文的核心洞见是：代理的性能不仅取决于底层模型能力，还高度依赖于接口设计的质量——好的 ACI 能显著降低代理出错的概率。
- **关键图表：** Figure 1（ACI 设计概览）、Figure 3（各接口设计的消融实验）、Table 1（SWE-bench 结果）
- **课程关联：** 模块 5（编程代理架构）——SWE-agent 是理解现代编程代理架构设计的核心论文，直接启发了 Claude Code 等产品级工具。

2. CodeAct: Executable Code Actions Elicit Better LLM Agents

- **完整引用：** Wang, X., Chen, Z., Lu, J., Bryan, K., Mishra, T., & Liu, P. (2024). Executable Code Actions Elicit Better LLM Agents. arXiv preprint arXiv: 2402.01030.
- **链接：** <https://arxiv.org/abs/2402.01030>
- **核心贡献：** CodeAct 提出了一种根本性的范式转变：用可执行的 Python 代码替代传统的 JSON/文本格式来表达代理动作。代理不再输出结构化的工具调用指令，而是直接编写并执行 Python 代码来完成任务。这种方式利用了代码的组合性和表达力，让代理可以在单次动作中实现复杂的逻辑控制、数据处理和多工具协调。实验证明 CodeAct 在多种代理任务上显著优于传统的 JSON action 格式。
- **关键图表：** Table 1（CodeAct vs JSON action 对比）、Table 2（不同任务的性能对比）
- **课程关联：** 模块 5（编程代理架构）——理解代理动作空间设计的权衡，代码作为通用行动语言的优势和局限。

3. Agentless: Demystifying LLM-based Software Engineering Agents

- **完整引用：** Xia, C.S., Deng, Y., Dunn, S., & Zhang, L. (2024). Agentless: Demystifying LLM-based Software Engineering Agents. arXiv preprint arXiv: 2407.01489.
- **链接：** <https://arxiv.org/abs/2407.01489>
- **核心贡献：** Agentless 对代理范式提出了尖锐的反思：通过两阶段流水线（定位 + 修复），不使用任何代理循环或工具调用，仅依赖 LLM 的直接推理能力，就能在 SWE-bench 上取得与复杂代理系统相当的性能。这项工作的价值在于为代理系统建立了一个简单而强大的 baseline，迫使研究者思考“代理化”带来的额外复杂性是否总是必要的。
- **关键图表：** Figure 1（两阶段流水线）、Table 1（与代理方法的性能对比）

- **课程关联：**模块 5（编程代理架构）——作为“反面教材”理解何时需要代理、何时简单方法已足够，培养架构选择的批判性思维。

搜索策略层（Search Strategy Layer）

搜索策略层论文探索了如何将**经典搜索算法**（如蒙特卡洛树搜索）与 **LLM 代理结合**，在更大的解空间中高效找到优质解。

论文	作者	年份	发表 Venue	核心贡献	推荐阅读章节
□ LATS: Language Agent Tree Search Unifies Reasoning, Acting, and Planning	Zhou et al.	2023	ICML 2024	将 MCTS 引入 LLM 代理框架	§ 3, § 4, Figure 1-2
SWE-Search: Enhancing Software Agents with Monte Carlo Tree Search and Iterative Refinement	Antonis et al.	2024	arXiv	将 MCTS 应用于软件工程代理	§ 3, § 4, Figure 1
CodeTree: Agent-guided Tree Search for Code Generation with Large	Li et al.	2024	arXiv	将树搜索专用于代码生成任务	§ 3, § 4, Figure 1

论文	作者	年份	发表 Venue	核心贡献	推荐阅读章节
Language Models					

论文摘要

1. LATS: Language Agent Tree Search Unifies Reasoning, Acting, and Planning

- **完整引用：** Zhou, A., Yan, K., Shlapentokh-Rothman, M., Wang, H., & Wang, Y.X. (2023). Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models. ICML 2024.
- **链接：** <https://arxiv.org/abs/2310.04406>
- **核心贡献：** LATS 首次将蒙特卡洛树搜索（MCTS）的核心思想引入 LLM 代理框架，将推理（Reasoning）、行动（Acting）和规划（Planning）统一在一个树搜索过程中。代理在每个决策点生成多个候选动作，通过环境反馈和自我评估进行节点评分，利用 UCB 公式平衡探索与利用，从而在复杂任务中系统性地搜索最优行动序列。LATS 在编程、网页交互和数学推理任务上均显著超越了 ReAct 和 Reflexion。
- **关键图表：** Figure 1（LATS 搜索过程图示）、Figure 2（与 ReAct/Reflexion 的对比）、Algorithm 1
- **课程关联：** 模块 4（搜索与规划策略）——核心论文，理解如何将经典 AI 搜索算法与现代 LLM 代理结合。

2. SWE-Search: Enhancing Software Agents with Monte Carlo Tree Search and Iterative Refinement

- **完整引用：** Antonis, A., et al. (2024). SWE-Search: Enhancing Software Agents with Monte Carlo Tree Search and Iterative Refinement. arXiv preprint arXiv: 2410.20285.
- **链接：** <https://arxiv.org/abs/2410.20285>
- **核心贡献：** SWE-Search 将 MCTS 方法专门应用于软件工程场景，针对代码仓库导航和 Bug 修复任务进行了定制化设计。该系统在搜索过程中结合了迭代精化机制，每个搜索节点代表一个代理状态（包括已浏览的文件、已尝试的修复等），通过 MCTS 在不同探索路径之间智能切换。论文验证了在复杂代码库中，系统性搜索比贪心策略更能找到正确的修复方案。
- **关键图表：** Figure 1（搜索树在代码仓库中的展开）、Table 1（SWE-bench 结果）
- **课程关联：** 模块 6（高级搜索策略）——理解搜索算法在真实软件工程任务中的具体应用和工程挑战。

3. CodeTree: Agent-guided Tree Search for Code Generation with Large Language Models

- **完整引用：** Li, J., et al. (2024). CodeTree: Agent-guided Tree Search for Code Generation with Large Language Models. arXiv preprint.
- **链接：** <https://arxiv.org/abs/2411.04329>
- **核心贡献：** CodeTree 专注于代码生成场景，将树搜索与代理引导的代码优化相结合。系统在搜索树的每个节点维护一个代码方案，通过测试用例反馈进行节点评估，并利用 LLM 代理来决定搜索方向（生成新方案、修改现有方案、或回溯尝试不同思路）。CodeTree 在 HumanEval 和 MBPP 等基准测试上取得了优异成绩，展示了搜索策略与代码特定启发式相结合的潜力。
- **关键图表：** Figure 1（CodeTree 框架图）、Table 1（与 baseline 的对比）
- **课程关联：** 模块 6（高级搜索策略）——作为搜索在代码生成领域的专业化应用案例。

多代理协作层（Multi-Agent Collaboration Layer）

多代理协作层论文探索了**多个 LLM 代理如何协作完成复杂任务**，包括角色分工、通信协议和工作流编排等核心问题。

论文	作者	年份	发表 Venue	核心贡献	推荐阅读章节
□ MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework	Hong et al.	2023	arXiv	用 SOP 规范化多代理协作流程	§ 3, § 4, Figure 1-3
AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation	Wu et al.	2023	arXiv	提出灵活的多代理对话框架	§ 3, § 4, Figure 1-2
ChatDev: Communicative Agents for	Qian et al.	2023	arXiv	模拟软件公司的代理协作模式	§ 3, § 4, Figure 1-2

论文	作者	年份	发表 Venue	核心贡献	推荐阅读章节
Software Development					

论文摘要

1. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework

- **完整引用：** Hong, S., Zhuge, M., Chen, J., Zheng, X., Cheng, Y., Zhang, C., Wang, J., Wang, Z., Yau, S.K.S., Lin, Z., Zhou, L., Ran, C., Xiao, L., Wu, C., & Schmidhuber, J. (2023). MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. arXiv preprint arXiv:2308.00352.
- **链接：** <https://arxiv.org/abs/2308.00352>
- **核心贡献：** MetaGPT 借鉴真实软件公司的标准化操作流程（SOP），为多代理系统引入了结构化的角色定义和工作流程规范。系统中的代理分别扮演产品经理、架构师、程序员、测试工程师等角色，通过标准化的中间产物（需求文档、设计文档、代码审查报告等）进行协作，而非自由对话。这种“元编程”方法有效减少了多代理系统中的幻觉级联问题（一个代理的错误被后续代理放大），显著提升了端到端的软件开发质量。
- **关键图表：** Figure 1（多角色工作流）、Figure 3（SOP 规范化流程）、Table 2（与 ChatDev 的对比）
- **课程关联：** 模块 7（多代理系统）——理解结构化协作如何优于自由对话，以及角色设计和信息流控制的重要性。

2. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation

- **完整引用：** Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., Awadallah, A.H., White, R.W., Burger, D., & Wang, C. (2023). AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv preprint arXiv:2308.08155.
- **链接：** <https://arxiv.org/abs/2308.08155>
- **核心贡献：** AutoGen 提出了一个高度灵活的多代理对话框架，其核心抽象是“可对话代理”（Conversable Agent）。每个代理可以是 LLM 驱动的、工具驱动的或人类参与的，代理之间通过多轮对话协作完成任务。AutoGen 的关键创新在于其灵活的对话模式——支持双代理对话、群组讨论、层级结构等多种拓扑，并能轻松集成人类反馈。该框架已成为多代理系统最广泛使用的开源实现之一。
- **关键图表：** Figure 1（可对话代理架构）、Figure 2（多种对话模式）、Table 1（应用场景）

- **课程关联：** 模块 7（多代理系统）——AutoGen 代表了“灵活对话式”的多代理设计思路，与 MetaGPT 的“结构化 SOP”形成互补。

3. ChatDev: Communicative Agents for Software Development

- **完整引用：** Qian, C., Cong, X., Yang, C., Chen, W., Su, Y., Xu, J., Liu, Z., & Sun, M. (2023). ChatDev: Communicative Agents for Software Development. arXiv preprint arXiv:2307.07924.
- **链接：** <https://arxiv.org/abs/2307.07924>
- **核心贡献：** ChatDev 将整个软件开发过程建模为一个虚拟软件公司中多个 LLM 代理的协作通信过程。代理分别担任 CEO、CTO、程序员、美术设计师、测试工程师等角色，通过聊天链（Chat Chain）顺序完成需求分析、技术设计、编码、美术资源生成、测试等阶段。ChatDev 的独特之处在于其“角色扮演”机制和“记忆流”设计，使代理间的对话更加聚焦和高效。该系统能在数分钟内从自然语言描述生成完整的可运行软件。
- **关键图表：** Figure 1（虚拟软件公司结构）、Figure 2（Chat Chain 流程）
- **课程关联：** 模块 7（多代理系统）——理解基于通信的多代理协作模式，以及角色扮演如何引导代理行为。

模型底座层（Model Foundation Layer）

模型底座层论文覆盖了**支撑代理能力的底层模型技术**，包括代码预训练、指令微调和安全对齐——这些是所有上层代理能力的基础。

论文	作者	年份	发表 Venue	核心贡献	推荐阅读章节
Evaluating Large Language Models Trained on Code (Codex)	Chen et al.	2021	arXiv	首次大规模评估代码生成模型	§ 2, § 3, Table 1-2
□ Training Language Models to Follow Instructions with Human Feedback	Ouyang et al.	2022	NeurIPS 2022	建立了人类反馈强化学习范式	§ 2, § 3, Figure 1-2

论文	作者	年份	发表 Venue	核心贡献	推荐阅读章节
(InstructGPT/RLHF)					
Constitutional AI: Harmlessness from AI Feedback	Bai et al.	2022	arXiv	提出基于 AI 反馈的自对齐方法	§ 2, § 3, Figure 1

论文摘要

1. Evaluating Large Language Models Trained on Code (Codex)

- **完整引用：** Chen, M., Tworek, J., Jun, H., Yuan, Q., de Oliveira Pinto, H.P., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., Ryder, N., Pavlov, M., Power, A., Kaiser, L., Bavarian, M., Winter, C., Tillet, P., Such, F.P., Cummings, D., Plappert, M., Chanez, F., Barnes, E., Herbert-Voss, A., Guss, W.H., Nichol, A., Paino, A., Tezak, N., Tang, J., Babuschkin, I., Balaji, S., Jain, S., Saunders, W., Hesse, C., Carr, A.N., Leike, J., Achiam, J., Misra, V., Morikawa, E., Radford, A., Knight, M., Brundage, M., Murati, M., Mayer, K., Welinder, P., McGrew, B., Amodei, D., McCandlish, S., Sutskever, I., & Zaremba, W. (2021). Evaluating Large Language Models Trained on Code. arXiv preprint arXiv:2107.03374.
- **链接：** <https://arxiv.org/abs/2107.03374>
- **核心贡献：** Codex 论文首次系统评估了在大规模代码语料上训练的语言模型的代码生成能力。论文提出了 HumanEval 基准测试——164 个手工编写的编程题目，成为代码生成领域最广泛使用的评估标准。Codex 模型能根据函数签名和文档字符串生成正确实现，论文还引入了 pass@k 评估指标来衡量通过多次采样找到正确解的概率。这项工作直接催生了 GitHub Copilot，开启了 AI 辅助编程的商业化时代。
- **关键图表：** Table 1（HumanEval 结果）、Table 2（pass@k 在不同 k 值下的表现）、Figure 3（模型规模与性能的关系）
- **课程关联：** 模块 1（代理基础）——理解代码生成模型的能力与局限，以及 pass@k 等评估方法如何影响后续搜索策略的设计。

2. Training Language Models to Follow Instructions with Human Feedback (InstructGPT/RLHF)

- **完整引用：** Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C.L., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens, M., Aspell, A., Welinder, P., Christiano, P., Leike, J., & Lowe, R. (2022). Training Language Models to Follow Instructions with Human Feedback. NeurIPS 2022.
- **链接：** <https://arxiv.org/abs/2203.02155>
- **核心贡献：** InstructGPT 论文建立了基于人类反馈的强化学习（RLHF）训练范式，通过三个阶段——监督微调（SFT）、奖励模型训练（RM）、近端策略优化（PPO）——将语言模型对齐到人类意图。这一方法使模型不仅能生成流畅的文本，还能准确理解和遵循人类指令，这对代理系统至关重要：一个不能可靠执行指令的模型无法成为可信赖的代理。RLHF 已成为所有主流 LLM（GPT-4、Claude、Gemini 等）的标准训练流程。
- **关键图表：** Figure 1（三阶段训练流程）、Figure 2（InstructGPT vs GPT-3 的人类偏好评估）
- **课程关联：** 模块 8（模型基础与对齐）——理解为什么当前 LLM 能作为代理工作的底层原因，以及指令跟随能力对代理可靠性的重要性。

3. Constitutional AI: Harmlessness from AI Feedback

- **完整引用：** Bai, Y., Kadavath, S., Kundu, S., Aspell, A., Kernion, J., Jones, A., Chen, A., Goldie, A., Mirhoseini, A., McKinnon, C., Chen, C., Olsson, C., Olah, C., Hernandez, D., Drain, D., Ganguli, D., Li, D., Tran-Johnson, E., Perez, E., Kerr, J., Mueller, J., Ladish, J., Landau, J., Ndousse, K., Lukosuite, K., Lovitt, L., Sellitto, M., Elhage, N., Schiefer, N., Mercado, N., DasSarma, N., Lasenby, R., Larson, R., Ringer, S., Johnston, S., Kravec, S., El Showk, S., Fort, S., Lanham, T., Telleen-Lawton, T., Conerly, T., Henighan, T., Hume, T., Bowman, S., Hatfield-Dodds, Z., Mann, B., Amodei, D., Joseph, N., McCandlish, S., Brown, T., & Kaplan, J. (2022). Constitutional AI: Harmlessness from AI Feedback. arXiv preprint arXiv: 2212.08073.
- **链接：** <https://arxiv.org/abs/2212.08073>
- **核心贡献：** Constitutional AI (CAI) 提出了一种基于 AI 自身反馈（而非人类标注）来实现模型对齐的方法。系统定义一组“宪法原则”（如“选择最无害的回复”），让 AI 根据这些原则对自身输出进行批评和修订（RLAIF 代替 RLHF 中的人类标注）。这一方法大幅降低了对齐的人工成本，同时实现了更一致、更可扩展的安全保障。对代理系统而言，CAI 的核心启示是：可以通过原则驱动的自我约束来确保代理行为安全——这与代理的工具使用安全和权限控制直接相关。

- **关键图表：** Figure 1（Constitutional AI 流程图）、Table 1（不同对齐方法的对比）
 - **课程关联：** 模块 8（模型基础与对齐）——理解代理安全的基础，以及如何通过原则和约束来规范代理行为。
-

扩展阅读

以下是按主题分类的补充论文和资源，供希望深入某一方向的同学参考。

代理框架与基准测试

- **SWE-bench: Can Language Models Resolve Real-World GitHub Issues?** (Jimenez et al., 2024) — 最重要的编程代理评估基准，收集了 2,294 个真实的 GitHub Issue 和对应的修复补丁，覆盖 12 个流行 Python 项目。所有评估均基于真实的测试套件，确保修复的正确性。后续的 SWE-bench Lite（300 个精选实例）和 SWE-bench Verified（500 个人工验证实例）进一步提升了评估质量。<https://arxiv.org/abs/2310.06770>
- **OpenHands: An Open Platform for AI Software Developers** (Wang et al., 2024) — 开源的 AI 软件开发平台，提供了代理开发的标准化框架，支持多种代理架构的快速实验和对比。<https://arxiv.org/abs/2407.16741>
- **AgentBench: Evaluating LLMs as Agents** (Liu et al., 2023) — 首个系统性的 LLM 代理能力评估基准，覆盖操作系统交互、数据库操作、知识图谱推理、数字卡牌游戏、横向思维谜题、家务模拟、网页浏览和网上购物共 8 个环境。<https://arxiv.org/abs/2308.03688>
- **HumanEval & MBPP** — 两个最基础的代码生成评估基准。HumanEval 包含 164 个手工编写的 Python 编程题；MBPP（Mostly Basic Programming Problems）包含 974 个由众包工人编写的入门级编程题。几乎所有代码生成论文都会报告这两个基准上的结果。

代码生成与理解

- **AlphaCode: Competition-Level Code Generation with AlphaCode** (Li et al., 2022) — DeepMind 的竞赛级代码生成系统。核心策略是大规模采样（百万级候选解）加智能过滤和聚类，在 Codeforces 编程竞赛中达到了参赛者中位数水平。这一“广度搜索”策略与后续的树搜索方法形成有趣对比。<https://arxiv.org/abs/2203.07814>

- **StarCoder: May the Source Be with You!** (Li et al., 2023) — BigCode 项目的开源代码 LLM，训练数据来源、处理流程和模型权重完全透明公开。15.5B 参数在 80+ 编程语言上训练，展示了开源社区在代码模型领域与商业模型竞争的可能性。<https://arxiv.org/abs/2305.06161>
- **SelfCodeAlign: Self-Alignment for Code Generation** (Wei et al., 2024) — 提出了一种无需人工标注的代码模型自对齐方法，让模型自己生成指令-代码对用于后续微调，实现了代码生成质量的自举提升。<https://arxiv.org/abs/2410.24198>
- **DeepSeek-Coder: When the Large Language Model Meets Programming** (Guo et al., 2024) — 开源代码模型系列，从 1.3B 到 33B 参数，在多个基准上达到或超越闭源模型。展示了在代码领域，精心设计的数据配比和训练策略可以大幅提升小模型的效率。<https://arxiv.org/abs/2401.14196>

推理与规划

- **LLM+P: Empowering Large Language Models with Optimal Planning Proficiency** (Liu et al., 2023) — 将 LLM 的自然语言理解能力与经典 PDDL 规划器的最优规划能力结合：LLM 负责将自然语言问题转化为形式化的规划问题描述，经典规划器负责求解，LLM 再将规划结果翻译回自然语言。<https://arxiv.org/abs/2304.11477>
- **Reasoning with Language Model is Planning with World Model** (Hao et al., 2023) — 提出 RAP (Reasoning via Planning) 框架，将 LLM 同时用作世界模型（预测行动结果）和推理代理，结合 MCTS 进行规划。这一工作在概念上连接了 LLM 推理与经典 AI 规划。<https://arxiv.org/abs/2305.14992>
- **Let's Verify Step by Step** (Lightman et al., 2023) — OpenAI 关于过程奖励模型 (Process Reward Model, PRM) 的重要论文，证明对推理的每一步进行验证比只看最终结果 (Outcome Reward Model, ORM) 更有效。这一发现对代理系统的搜索策略设计有直接启示：逐步评估优于整体评估。<https://arxiv.org/abs/2305.20050>

多代理与社会模拟

- **Generative Agents: Interactive Simulacra of Human Behavior** (Park et al., 2023) — 在一个类似「The Sims」的沙盒环境中部署了 25 个 LLM 驱动的代理，每个代理拥有独立的记忆流、反思能力和规划能力。这些代理自发产生了复杂的社会行为，如组织情人节派对、传播信息、形成社交关系等。这项工作展示了 LLM 代理在开放式环境中的涌现行为。<https://arxiv.org/abs/2304.03442>

- **Scaling Instructable Agents Across Many Simulated Worlds (SIMA)** (Reed et al., 2024) — DeepMind 的多世界可指令代理，能在多种 3D 游戏环境中理解和执行自然语言指令。
- **Voyager: An Open-Ended Embodied Agent with Large Language Models** (Wang et al., 2023) — 在 Minecraft 中实现的开放式具身代理，具有自动课程学习、技能库构建和代码生成能力，能持续探索和学习新技能。<https://arxiv.org/abs/2305.16291>

安全、对齐与可信代理

- **Toolemu: Identifying the Risks of LM Agents with an LM-Emulated Sandbox** (Ruan et al., 2023) — 提出用 LLM 模拟工具执行环境来评估代理的安全风险，发现当前代理在工具使用时存在大量安全隐患。<https://arxiv.org/abs/2309.15817>
 - **R-Judge: Benchmarking Safety Risk Awareness for LLM Agents** (Yuan et al., 2024) — 构建了首个评估代理安全风险意识的基准，测试代理能否识别和拒绝有害的行动序列。<https://arxiv.org/abs/2401.10019>
 - **The Landscape of Emerging AI Agent Architectures for Reasoning, Planning, and Tool Calling** (Masterman et al., 2024) — 一篇优秀的综述论文，系统梳理了 AI 代理架构的设计空间。推荐作为课程入门综述阅读。<https://arxiv.org/abs/2404.11584>
-

阅读建议

推荐阅读顺序

建议按以下顺序循序渐进地阅读论文，以建立完整的知识体系：

第一阶段：建立基础（第 1-2 周） 1. **CoT** → 理解 LLM 推理的起点 2. **ReAct** → 理解代理循环的核心范式 3. **Toolformer** → 理解工具使用机制

第二阶段：深入反思与搜索（第 3-4 周） 4. **Self-Refine** → 理解自我迭代优化 5. **Reflexion** → 理解经验学习机制 6. **Tree of Thoughts** → 从线性推理过渡到搜索

第三阶段：编程代理实践（第 5-6 周） 7. **SWE-agent** → 掌握代理接口设计 8. **CodeAct** → 理解代码作为动作空间 9. **Agentless** → 建立批判性视角

第四阶段：高级主题（第 7-8 周） 10. **LATS** → 掌握搜索策略与代理的结合 11. **MetaGPT** → 理解多代理协作 12. **AutoGen / ChatDev** → 了解不同协作范式

第五阶段：底层原理（贯穿全程，按需深入） 13. **Codex** → 代码模型能力的基线 14. **InstructGPT** → 指令跟随的训练方法 15. **Constitutional AI** → 安全与对齐原则

高效阅读策略

- 首次阅读（30 分钟）：** 只读摘要（Abstract）、引言（Introduction）的最后一段（通常包含贡献总结）、方法章节的第一段和配图、结论（Conclusion）。目标：理解论文做了什么、为什么做、核心方法是什么。
- 重点精读（1-2 小时）：** 根据上文“推荐阅读章节”，精读方法部分的核心内容，对照关键图表理解技术细节。目标：能用自己的话向他人解释核心方法。
- 实践验证（2-4 小时）：** 查看论文的官方代码仓库，运行关键实验或示例代码。对于编程代理类论文（SWE-agent、CodeAct 等），建议亲手搭建并测试。目标：将论文知识转化为实践能力。
- 横向对比：** 将同一层内的论文进行对比阅读，关注它们解决了什么共同问题、各自的优劣和适用场景。这比逐篇孤立阅读效果好得多。

阅读笔记模板

建议为每篇论文撰写结构化笔记，以下为推荐模板：

论文标题：

阅读日期：

一句话总结：（用自己的话，不超过 30 字）

核心问题：这篇论文要解决什么问题？

核心方法：用了什么方法？有哪些关键创新？

核心结果：最重要的实验结果是什么？

与其他论文的关系：

- 继承了哪些前序工作的思路？
- 与同层其他论文有何异同？
- 启发了哪些后续工作？

对课程项目的启示：

- 哪些设计可以直接应用？
- 有哪些局限性需要注意？

开放问题：阅读后仍有哪些疑问？

论文间的关键对比维度

在横向对比论文时，建议关注以下几个维度：

对比维度	说明
推理方式	线性（CoT） vs 树状（ToT） vs 图状（LATS）
行动格式	自然语言 vs JSON 结构化 vs 可执行代码（CodeAct）
反馈来源	环境反馈 vs 自我反馈 vs 人类反馈
记忆机制	无记忆 vs 短期（上下文内） vs 长期（Reflexion）
代理数量	单代理 vs 多代理协作
搜索策略	贪心（ReAct） vs 系统搜索（LATS, SWE-Search）
代理化程度	无代理（Agentless） vs 轻量代理 vs 完全自主代理

必读论文清单（最小集合）

如果时间有限，以下 6 篇为绝对必读（标记为 □ 的论文）：

- 1. **CoT** — 理解推理基础
- 2. **ReAct** — 理解代理范式
- 3. **Reflexion** — 理解反思机制
- 4. **SWE-agent** — 理解编程代理设计
- 5. **LATS** — 理解搜索策略
- 6. **MetaGPT** — 理解多代理协作

这 6 篇论文覆盖了从基础到高级的完整链路，阅读后即可建立对 AI 编程代理领域的系统性理解。

常见术语对照表

以下对照表帮助你在阅读英文论文时快速理解关键术语：

英文术语	中文翻译	首次出现论文
Chain-of-Thought (CoT)	思维链	Wei et al., 2022
Reasoning-Acting Loop	推理-行动循环	Yao et al. (ReAct), 2022

英文术语	中文翻译	首次出现论文
Agent-Computer Interface (ACI)	代理-计算机接口	Yang et al. (SWE-agent), 2024
Verbal Reinforcement Learning	语言强化学习	Shinn et al. (Reflexion), 2023
Test-time Compute Scaling	推理时计算扩展	Yao et al. (ToT), 2023
Monte Carlo Tree Search (MCTS)	蒙特卡洛树搜索	经典 AI，被 LATS 引入 LLM 领域
Standard Operating Procedure (SOP)	标准操作流程	Hong et al. (MetaGPT), 2023
Conversable Agent	可对话代理	Wu et al. (AutoGen), 2023
Process Reward Model (PRM)	过程奖励模型	Lightman et al., 2023
pass@k	通过率@k（采样 k 次的通过率）	Chen et al. (Codex), 2021
RLHF	基于人类反馈的强化学习	Ouyang et al. (InstructGPT), 2022
RLAIF	基于 AI 反馈的强化学习	Bai et al. (Constitutional AI), 2022